

LegisTrack — Guía de presentación

Cámara de Diputados de Argentina: predictor de votaciones

Estefanía Zangaro · Martina Pusso · Milagros Cosentino · Juan Manuel Maiorano

El proyecto

LegisTrack predice cómo votaría cada diputado activo de la Cámara ante un proyecto de ley hipotético. El usuario ingresa el título de una ley y el sistema devuelve una predicción por diputado (AFIRMATIVO, NEGATIVO o ABSTENCIÓN), con una estimación de si el proyecto lograría la mayoría simple. El modelo se entrena sobre el historial real de votaciones nominales de la cámara.

Cómo se encadenan los notebooks

Cada etapa lee un archivo, lo transforma y guarda el resultado para que la siguiente lo use. Los datos originales nunca se modifican.

Scraping.ipynb	→	hcdn_votaciones_historico.csv	(~578.500 filas)
STG_1_Filtrado.ipynb	→	votaciones_filtrado.csv	(solo 257 diputados activos)
STG_2_transformacion	→	df_consolidado.csv	(1 voto por diputado por ley)
STG_3_filtro_titulos	→	df_modelado.csv	(~28.700 filas útiles)
STG_4_features_titulo	→	df_features_titulo.csv	(1.022 títulos con tema)

Scraping — cómo conseguimos los datos

Qué hace: consulta la API pública de la HCDN votación por votación, descarga el resultado de cada diputado en cada sesión y acumula todo en una sola tabla. El archivo resultante tiene una fila por voto individual: nombre del diputado, bloque, provincia, título del proyecto y resultado (AFIRMATIVO, NEGATIVO, ABSTENCIÓN).

Por qué fue necesario: los datos de la cámara son públicos pero no están disponibles en un solo archivo descargable. El scraping automatiza lo que de otra forma habría requerido copiar miles de registros a mano.

Guarda para la siguiente etapa: hcdn_votaciones_historico.csv — el historial completo, sin ningún filtro.

Pregunta frecuente: *¿Por qué no usaron un Excel que ya existía?* No existe un dataset oficial consolidado. La única fuente completa es la API de la HCDN.

STG 1 — Filtrado al padrón actual

Qué hace: compara el historial completo contra la lista de los 257 diputados que integran la cámara hoy y descarta todos los registros de personas que ya no tienen mandato. Para poder hacer esa comparación, normaliza los nombres (minúsculas, sin tildes, sin espacios extra), porque el mismo diputado puede aparecer escrito de formas distintas en distintas fuentes. Con esa normalización se logró identificar los 257 diputados actuales sin correcciones manuales.

Por qué fue necesario: el historial abarca varios períodos. Un modelo que predice a los diputados actuales no debería entrenarse con los votos de quienes ya no están.

Guarda para la siguiente etapa: votaciones_filtrado.csv — solo los votos de los 257 diputados activos.

Pregunta frecuente: ¿Los votos de ex diputados se pierden? No, siguen en el archivo crudo. Solo se excluyen del entrenamiento porque no los vamos a predecir.

STG 2 — Transformación y consolidación

Qué hace: resuelve dos problemas del historial crudo.

El primero: cuando una ley se vota artículo por artículo, el dataset tiene una fila por artículo. Un diputado puede tener 20 filas para la misma ley. El notebook detecta cuándo hubo un «voto en general» (postura sobre el proyecto completo) y le da prioridad. Si solo hay votos por artículo, toma el más frecuente. En caso de empate, asigna ABSTENCIÓN. El resultado es una sola postura por diputado por ley.

El segundo: el mismo proyecto aparece bajo títulos distintos («Ley de Educación — Art. 1», «Ley de Educación — Dictamen de Mayoría», etc.). El notebook extrae el «título base» de cada proyecto, eliminando esos sufijos, para que todas las variantes del mismo proyecto queden unificadas bajo un solo nombre.

Por qué fue necesario: sin consolidación, el modelo vería decenas de filas repetidas por ley y diputado. Sin unificación de títulos, trataría cada variante como una ley distinta.

Guarda para la siguiente etapa: df_consolidado.csv — una fila por diputado por proyecto, con bloque, provincia, voto y fecha.

Pregunta frecuente: ¿Por qué no se promedia el voto? El voto es una categoría (AFIRMATIVO, NEGATIVO, ABSTENCIÓN), no un número. No se puede promediar; se toma el valor más frecuente.

STG 3 — Filtro de títulos sin valor temático

Qué hace: identifica y elimina las votaciones que no corresponden a proyectos de ley con contenido. En la cámara también se votan mociones de orden, habilitaciones de sesión, solicitudes de licencia y otros trámites internos. Para cada título del dataset, el notebook evalúa si describe el contenido de una ley o si es un trámite procedimental. Los que no describen ninguna ley se eliminan. El proceso se verifica con 36 casos de prueba automáticos para asegurar que el filtro no borre leyes reales.

Por qué fue necesario: entrenar el modelo con registros procedimentales sería como enseñarle a predecir votos mirando cómo se organiza la sesión, no qué se discute.

Guarda para la siguiente etapa: df_modelado.csv — 28.700 filas y 1.022 títulos únicos de leyes con contenido real. Este es el archivo base para el modelo.

Pregunta frecuente: ¿Cómo saben que el filtro no borra leyes reales? Con los 36 casos de prueba: si alguno falla, el notebook avisa antes de guardar.

STG 4 — Features semánticas de los títulos

Qué hace: le da al modelo una forma de entender de qué trata cada ley. Toma los 1.022 títulos únicos y los convierte en representaciones numéricas usando un modelo de lenguaje entrenado en español. Esas representaciones capturan el significado del texto: dos títulos que hablan del mismo tema quedan «cerca» aunque usen palabras distintas («régimen jubilatorio» y «sistema previsional» son distintos en texto pero similares en significado). Con esas representaciones, agrupa los 1.022 títulos en 20 categorías temáticas (salud, educación, infraestructura, legislación laboral, etc.).

Por qué fue necesario: sin información temática, el modelo no podría detectar que un diputado vota sistemáticamente en contra de ciertos temas y a favor de otros. El tema de la ley es una de las variables más importantes para predecir el voto.

Guarda para la siguiente etapa: `df_features_titulo.csv` — cada título con su categoría temática asignada. Este archivo se combinará con los datos del diputado para armar el dataset de entrenamiento.

Pregunta frecuente: *¿Por qué 20 grupos?* Se probaron 10, 15 y 20. Con 10 los grupos mezclaban temas muy distintos; con 20 cada grupo era coherente y describible.

App Streamlit — lo que puede hacer hoy

Qué hace: permite consultar el historial de votaciones de cualquier diputado activo. El usuario elige un nombre en un selector, presiona «Consultar» y ve: el bloque al que pertenece, la provincia que representa, el conteo total de votos por tipo (AFIRMATIVO, NEGATIVO, ABSTENCIÓN) y las últimas 10 votaciones con título, fecha y resultado. La funcionalidad de predicción aparecerá cuando el modelo esté entrenado.

Lee: `df_consolidado.csv`.

Pregunta frecuente: *¿Por qué no tiene predicción todavía?* La predicción requiere el modelo, que es la etapa que sigue. Se construyó la app primero para tener algo funcional para mostrar.

Deploy en Streamlit Cloud

Qué es: Streamlit Community Cloud es un servicio gratuito que publica la app en internet a partir del repositorio de GitHub. Cada vez que se sube un cambio al repositorio, la app se actualiza automáticamente.

Cómo se hizo: se conectó el repositorio de GitHub con una cuenta de Streamlit Cloud y se indicó cuál archivo es la app (`app/app.py`). Streamlit Cloud lee el `requirements.txt` (la lista de librerías que necesita la app), las instala y pone la app en línea.

Problema que tuvimos: las versiones fijas de las librerías no eran compatibles con la versión de Python de Streamlit Cloud. Se resolvió sacando los números de versión del `requirements.txt` para que el servicio elija las versiones más recientes disponibles.

Link: votos-diputados-gxuaqsn2fp3jdtelqxfgy.streamlit.app

Pregunta frecuente: *¿Hay que pagar?* No. Streamlit Community Cloud es gratuito para proyectos públicos conectados a GitHub.
